

Hooks — the out-of-scope fence, enforced

Day 3 teaches **blast-radius control**: before the agent edits anything, you say what it must not touch. A worksheet records that as a promise. A **hook** makes it a wall.

That gap is the lesson. An agent that has *agreed* not to touch `ingest.py` will still touch `ingest.py` on turn nine, when the context has filled and the original constraint is competing for attention with forty turns of noise. (That is Day 2's Context Saturation, and Day 3 does not cure it.) A `PreToolUse` hook does not forget.

What ships here

Script	Fires on	Blocks
<code>protect_key.sh</code>	<code>Read/Edit/Write/Grep/Glob/Bash</code>	Any read of <code>.env</code> ; any Bash command that would print it (<code>cat</code> , <code>grep</code> , <code>cp</code> , <code>base64</code> , <code>redirects...</code>)
<code>protect_files.sh</code>	<code>Edit/Write</code>	Edits to <code>guardrails.py</code> , <code>self_critique.py</code> , <code>ingest.py</code> , <code>config.py</code> , <code>offline_stubs.py</code>
<code>block_dangerous_command.sh</code>	<code>Bash</code>	<code>rm -rf</code> , <code>sudo</code> , <code>force-push</code> , <code>git clean -fdx</code> , <code>curl sh</code> , <code>fork bombs</code> — and <code>pip install</code>

They are wired up in `.claude/settings.json`.

The key rule: use the secret, never see it

`protect_key.sh` does **not** hide the key from your program. It hides it from the *agent*. Your code reads it the normal way:

```
import os
from dotenv import load_dotenv    # pip install python-dotenv

load_dotenv()                    # loads .env from the project root
API_KEY = os.getenv("GEMINI_API_KEY")
```

So the agent can still prove the key loads, without ever seeing its value:

```
python3 -c "import os, dotenv; dotenv.load_dotenv();
print(bool(os.getenv('GEMINI_API_KEY')))"
```

```
# -> True
```

That command is **allowed**. `cat .env` is **denied**. `.env.example` is allowed, because there is nothing in it to leak.

Why `pip install` is on the blacklist

The Day 3 fence says *no new dependency — no cross-encoder, no sentence-transformers*. An agent told to "improve retrieval quality" reaches for a reranker model within about two turns; it is the single most likely way a learner's run escapes the fence and stops being comparable to their partner's.

Blocking the install turns that sentence into a wall, and the denial message becomes the teaching moment. `pip install -r requirements.txt` is still allowed.

The limitation, said out loud

A path-based hook cannot protect a **function**. `build_graph()` lives inside `agent.py`, which learners *must* edit for slice 2, so "don't rewire the graph" cannot be enforced here. It stays a human check — `git diff RAG/agent.py` in the L3 proof step.

Fences are cheap where the unit of protection matches the unit of enforcement. Where it doesn't, you still have to look.

Turning them on

Project hooks load from `.claude/settings.json` when Claude Code's **project directory** is `day3-sandbox/`. Open that folder — not its parent, not `RAG/` — or the hooks will not fire.

```
cd day3-sandbox
claude          # or: VS Code -> File -> Open Folder -> day3-sandbox
```

Claude Code only watches directories that had a settings file when the session started. If you added these mid-session, open `/hooks` once (that reloads the config) or restart.

Review, edit, or disable them any time with the `/hooks` command. Silent success is invisible by design — the UI only reports hooks that error or run slowly.

Testing a hook without an agent

Hooks read a JSON payload on stdin. Pipe one in by hand:

```
cd day3-sandbox

# should print a deny decision
echo '{"tool_name":"Read","tool_input":{"file_path":"RAG/.env"}}' | bash
hooks/protect_key.sh
```

```
# should print nothing (allowed)
echo '{"tool_name":"Read","tool_input":{"file_path":"RAG/agent.py"}}' | bash
hooks/protect_key.sh

# should print a deny decision citing the fence
echo '{"tool_name":"Bash","tool_input":{"command":"pip install sentence-
transformers"}}' \
| bash hooks/block_dangerous_command.sh
```

Silence means allow. A JSON body on stdout means deny. Every script exits 0 either way — the decision travels in the JSON, not the exit code:

```
{"hookSpecificOutput":{"hookEventName":"PreToolUse",
                        "permissionDecision":"deny",
                        "permissionDecisionReason":"...shown to the agent..."}}
```

The `permissionDecisionReason` is fed back to the agent, so write it as an instruction, not an error. "Re-ranking must land in `rerank.py`" redirects the agent; "Permission denied" makes it retry.

Watch-out when writing your own: if you embed Python via a heredoc (`python3 - <<'PY'`), the heredoc is Python's stdin — `json.load(sys.stdin)` reads the script, not the payload, and your hook silently allows everything. These scripts pass the payload in `CLAUDE_HOOK_INPUT` instead. Always pipe-test a new hook before you trust it.

Belt and braces

`.claude/settings.json` also carries a declarative deny rule:

```
"permissions": { "deny": [ "Read(./RAG/.env)", "Read(./env)" ] }
```

Permission rules are simpler and harder to get wrong; hooks are scriptable and can inspect a Bash command's *contents*. Secrets are worth both.